数据结构：图算法
复习讲义

目录

1 图的基本定义 (Graph Definitions) 2
1.1 图、顶点与边 (Graph, Vertices & Edges) . . . 2
1.2 完全图与边数 (Complete Graph & Number of Edges) . . . 2
1.3 相邻、关联与子图 (Adjacency, Incidence & Subgraph) . . . 2
1.4 路径、环与连通 (Path, Cycle & Connectivity) . . . 3
1.5 连通分量、树与有向连通性 . . . 3
1.6 度、入度与出度 (Degree, In-degree & Out-degree) . . . 3

2 图的存储表示 (Representation of Graphs) 4
2.1 邻接矩阵 (Adjacency Matrix) . . . 4
2.2 邻接表 (Adjacency Lists) . . . 5
2.3 逆邻接表与十字链表 . . . 5
2.4 邻接多重表与带权图 (Adjacency Multilists & Weighted Edges) . . . 6

3 AOV 网络与偏序关系 (AOV Network & Partial Order) 6
3.1 课程先修关系示例 . . . 6
3.2 AOV 网络 (Activity-On-Vertex Network) . . . 7
3.3 前驱、后继与偏序 . . . 7

4 拓扑排序 (Topological Sort) 8
4.1 拓扑序的定义 . . . 8
4.2 课程例子的一个拓扑序 . . . 8
4.3 拓扑排序目标 . . . 9
4.4 朴素算法：每次重新寻找入度为 0 的顶点 . . . 9
4.5 改进算法：维护所有入度为 0 的顶点 . . . 9
4.6 算法直觉与正确性 . . . 10

5 补充知识点与易错点 **11**
5.1 邻接矩阵与邻接表的选择 11
5.2 拓扑排序常考结论 11
5.3 DFS 版拓扑排序 11

6 最短路径问题的基本概念 **12**
6.1 带权路径长度 12
6.2 单源最短路径问题 12
6.3 表结构 Table 12
6.4 松弛操作 12

7 无权最短路径 (Unweighted Shortest Paths) **13**
7.1 基本思想 13
7.2 简单实现 13
7.3 队列改进实现 14
7.4 课件例子的最终表 14

8 Dijkstra 算法 **15**
8.1 适用范围与集合定义 15
8.2 贪心选择规则 15
8.3 算法伪代码 15
8.4 课件例子的最终表 16
8.5 路径打印 16
8.6 Dijkstra 的两种实现 17

9 带负权边的图 **17**
9.1 为什么不能简单加常数 17
9.2 队列式负权算法 17
9.3 复杂度与负权环 18

10 有向无环图中的最短路径 **18**
10.1 按拓扑序处理 18
10.2 AOE 网络 19
10.3 EC 与 LC 的计算 19
10.4 课件 AOE 例题 19

11 全源最短路径问题 **20**
11.1 问题定义 20
11.2 课件给出的两种方法 21

12 最短路径算法选择总结 **21**

13 网络流问题 (Network Flow Problems) **21**
13.1 最大流模型 21
13.2 增广路算法的基本版本 22
13.3 残量图与反向边 22
13.4 整数容量下的复杂度 23
13.5 选择增广路的改进策略 23
13.6 补充说明：特殊容量与最小费用流 24

14 最小生成树 (Minimum Spanning Tree) **24**
14.1 生成树与最小生成树 24
14.2 贪心方法的约束 24
14.3 Prim 算法：grow a tree 25
14.4 Kruskal 算法：maintain a forest 25
14.5 Prim 与 Kruskal 对比 26

15 深度优先搜索的应用 (Applications of DFS) **26**
15.1 DFS 模板 26
15.2 利用 DFS 求连通分量 (Finding Connected Components) 27
15.3 无向图中的 DFS (Undirected Graphs) 28
15.4 双连通性 (Biconnectivity) 28
15.5 欧拉回路 (Euler Circuits) 29

16 全章总结 (Summary) **29**

1 图的基本定义 (Graph Definitions)

1.1 图、顶点与边 (Graph, Vertices & Edges)

图通常记为 $G(V, E)$，其中：

- G **(Graph)**：图。
- $V = V(G)$ **(Vertices)**：有限非空顶点集合。
- $E = E(G)$ **(Edges)**：有限边集合。

无向图与有向图

- **无向图 (Undirected Graph)**：边没有方向，$(v_i, v_j) = (v_j, v_i)$ 表示同一条边。
- **有向图 (Directed Graph / Digraph)**：边有方向，记作 $\langle v_i, v_j \rangle$，其中 v_i 是尾 (tail)，v_j 是头 (head)，一般有 $\langle v_i, v_j \rangle \neq \langle v_j, v_i \rangle$。

本章默认限制

1. **自回路 (Self Loop) 不允许**：即不考虑 (v_i, v_i) 或 $\langle v_i, v_i \rangle$。
2. **重图 (Multigraph) 不考虑**：即同一对顶点之间不允许出现多条平行边。

1.2 完全图与边数 (Complete Graph & Number of Edges)

完全图 (Complete Graph) 是边数达到最大值的图。

完全图边数公式

若 $|V| = n$：

- **无向完全图**：任意两个不同顶点之间有一条无向边，

$$|E| = C_n^2 = \frac{n(n-1)}{2}.$$

- **有向完全图**：任意两个不同顶点之间两个方向都可有边，

$$|E| = P_n^2 = n(n-1).$$

1.3 相邻、关联与子图 (Adjacency, Incidence & Subgraph)

- 在无向图中，若存在边 (v_i, v_j)，则 v_i 与 v_j **相邻 (adjacent)**；边 (v_i, v_j) **关联 (incident on)** 于 v_i 和 v_j。

- 在有向图中，若存在边 $\langle v_i, v_j \rangle$，则 v_i **adjacent to** v_j，v_j **adjacent from** v_i；该有向边关联于 v_i 和 v_j。
- **子图 (Subgraph)**：$G' \subseteq G$ 当且仅当

$$V(G') \subseteq V(G) \quad \text{且} \quad E(G') \subseteq E(G).$$

1.4 路径、环与连通 (Path, Cycle & Connectivity)

- **路径 (Path)**：从 v_p 到 v_q 的路径是顶点序列

$$\{v_p, v_{i1}, v_{i2}, \ldots, v_{in}, v_q\},$$

  使得相邻顶点之间的边都属于 $E(G)$。在有向图中，边的方向必须沿着路径方向。
- **路径长度 (Length of a Path)**：路径上边的条数。
- **简单路径 (Simple Path)**：路径中除可能的首尾外，其余顶点 $v_{i1}, v_{i2}, \ldots, v_{in}$ 互不相同。
- **环 (Cycle)**：首尾相同的简单路径，即 $v_p = v_q$。
- **无向图中的连通 (Connected)**：若从 v_i 到 v_j 有路径，则 v_i 和 v_j 连通；无向图中这也意味着从 v_j 到 v_i 有路径。
- **连通图 (Connected Graph)**：无向图中任意一对不同顶点都连通。

1.5 连通分量、树与有向连通性

- **连通分量 ((Connected) Component)**：无向图中的极大连通子图。
- **树 (Tree)**：连通且无环的图。
- **DAG (Directed Acyclic Graph)**：有向无环图。
- **强连通有向图 (Strongly Connected Directed Graph)**：对 $V(G)$ 中任意 v_i, v_j，既存在从 v_i 到 v_j 的有向路径，也存在从 v_j 到 v_i 的有向路径。
- **弱连通 (Weakly Connected)**：若忽略有向边方向后图是连通的，则称该有向图弱连通。
- **强连通分量 (Strongly Connected Component)**：极大强连通子图。

1.6 度、入度与出度 (Degree, In-degree & Out-degree)

- **度 (Degree)**：与顶点 v 关联的边数，记为 $\mathrm{degree}(v)$。
- **有向图**中还要区分：
  - **入度 (In-degree)**：指向 v 的边数。

- **出度 (Out-degree)**：从 v 指出的边数。
- 总度数通常为 $\text{degree}(v) = \text{in-degree}(v) + \text{out-degree}(v)$。

若无向图 G 有 n 个顶点、e 条边，并令 $d_i = \text{degree}(v_i)$，则：

$$e = \frac{1}{2}\sum_{i=0}^{n-1} d_i.$$

这就是握手定理：每条无向边会被两个端点各计数一次。

2 图的存储表示 (Representation of Graphs)

2.1 邻接矩阵 (Adjacency Matrix)

对含 n 个顶点的图 $G(V,E)$，邻接矩阵 `adj_mat[n][n]` 定义为：

$$\texttt{adj_mat}[i][j] = \begin{cases} 1, & (v_i, v_j) \in E \text{ 或} \langle v_i, v_j \rangle \in E, \\ 0, & \text{otherwise.} \end{cases}$$

- **无向图**的邻接矩阵是对称矩阵，因为 $(v_i, v_j) = (v_j, v_i)$。
- **判断某条边是否存在**的时间为 $O(1)$。
- **遍历所有边**通常要扫描整个矩阵，时间为 $O(n^2)$。
- **空间复杂度**为 $O(n^2)$，因此邻接矩阵适合稠密图，不适合顶点很多但边很少的稀疏图。

无向图矩阵压缩存储

由于无向图的邻接矩阵对称，可以只保存下三角部分。若使用 1-based 下标并保存

$$\{a_{11}, a_{21}, a_{22}, \ldots, a_{n1}, \ldots, a_{nn}\},$$

则需要 $n(n+1)/2$ 个位置。对 $i \geq j$，a_{ij} 在一维数组中的下标可写为：

$$\text{index}(a_{ij}) = \frac{i(i-1)}{2} + j.$$

用邻接矩阵求度

- 无向图：

$$\text{degree}(i) = \sum_{j=0}^{n-1} \texttt{adj_mat}[i][j].$$

- 有向图：

$$\text{out-degree}(i) = \sum_{j=0}^{n-1} \texttt{adj_mat}[i][j], \qquad \text{in-degree}(i) = \sum_{j=0}^{n-1} \texttt{adj_mat}[j][i].$$

2.2 邻接表 (Adjacency Lists)

邻接表的思想是：把邻接矩阵的每一行替换为一个链表。链表 `graph[i]` 中存放从顶点 i 出发可以到达的邻接点。

顶点	邻接表	含义
0	1	$0 \to 1$
1	0, 2	$1 \to 0,\ 1 \to 2$
2	空	顶点 2 没有出边

上表对应的邻接矩阵为：

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}.$$

邻接表中每个链表结点的顺序不影响图的含义。

邻接表的空间与时间

- 无向图中，每条边 (i, j) 会在 `graph[i]` 和 `graph[j]` 中各出现一次，因此需要 n 个表头和 $2e$ 个链表结点。课件中的空间估计写作：

$$S = n \text{ heads} + 2e \text{ nodes} = (n + 2e) \text{ ptrs} + 2e \text{ ints}.$$

- 遍历所有顶点和边的时间为：

$$T = O(n + e).$$

- 无向图中，$\text{degree}(i)$ 等于 `graph[i]` 中的结点个数。
- 有向图中，`graph[i]` 的长度直接给出出度；若要快速得到入度，可以增加逆邻接表。

2.3 逆邻接表与十字链表

有向图中求入度时，仅靠普通邻接表需要扫描所有链表。课件给出两种处理方式：

1. **增加逆邻接表 (Inverse Adjacency Lists)**：`inv[j]` 存放所有指向 j 的顶点。以上一节例子为例：

逆邻接表	内容
`inv[0]`	1
`inv[1]`	0
`inv[2]`	1

2. **十字链表 / 多重链表思想 (Multilist)**：每条有向边 $\langle i, j \rangle$ 的结点同时记录 `tail i` 与 `head j`，并同时挂到“以 i 为尾”的行链和“以 j 为头”的列链上，从而兼顾出边和入边访问。

2.4 邻接多重表与带权图 (Adjacency Multilists & Weighted Edges)

在无向图的普通邻接表中，边 (i, j) 会出现两个结点：一个在 `graph[i]` 中记录 j，一个在 `graph[j]` 中记录 i。邻接多重表将这两个结点合并为一个边结点。

邻接多重表结点的典型字段

一个边结点通常包含：

- **mark**：标记该边是否已经被访问或处理。
- v_1, v_2：边的两个端点。
- **next** v_1**, next** v_2：分别指向与 v_1、v_2 相关的下一条边。

虽然邻接多重表字段更多，但它的好处是：当算法需要**标记一条边**并继续找到下一条相关边时，可以直接围绕同一个边结点操作，而不必维护两份重复结点的一致性。

带权图只需在存储结构中增加权值：

- 邻接矩阵：令 `adj_mat[i][j] = weight`，非边位置可用 0、∞ 或特殊标记表示，具体取决于算法。
- 邻接表 / 邻接多重表：在边结点中增加 `weight` 字段。

3 AOV 网络与偏序关系 (AOV Network & Partial Order)

3.1 课程先修关系示例

课件用一个假想大学的计算机科学学位课程作为例子。每门课程是一个活动，先修关系决定了有向边。

Course number	Course name	Prerequisites
C1	Programming I	None
C2	Discrete Mathematics	None
C3	Data Structure	C1, C2
C4	Calculus I	None
C5	Calculus II	C4
C6	Linear Algebra	C5
C7	Analysis of Algorithms	C3, C6
C8	Assembly Language	C3
C9	Operating Systems	C7, C8
C10	Programming Languages	C7
C11	Compiler Design	C10
C12	Artificial Intelligence	C7
C13	Computational Theory	C7
C14	Parallel Algorithms	C13
C15	Numerical Analysis	C6

把这张表转换成图时，规则是：若课程 C_i 是课程 C_j 的先修课，则建立有向边 $C_i \to C_j$。

3.2 AOV 网络 (Activity-On-Vertex Network)

AOV Network 是一个有向图 G：

- $V(G)$ 表示活动，例如课程。
- $E(G)$ 表示优先关系，例如先修课关系。
- 例如 $C1 \to C3$ 表示 $C1$ 是 $C3$ 的先修课程。

3.3 前驱、后继与偏序

- i **是** j **的前驱 (predecessor)**：存在从 i 到 j 的路径。
- i **是** j **的直接前驱 (immediate predecessor)**：$\langle i, j\rangle \in E(G)$。
- 若 i 是 j 的前驱，则 j 是 i 的后继 (successor)；若边直接存在，则称为直接后继。

偏序 (Partial Order) 是同时满足以下性质的先后关系：

- **传递性 (Transitive)**：若 $i \to k$ 且 $k \to j$，则 $i \to j$。
- **反自反性 (Irreflexive)**：$i \to i$ 不可能成立。

AOV 可行性的核心判断

如果先后关系是自反的，就意味着某个活动 i 必须在自己开始前完成，即 i 是自己的前驱。这是不可能完成的。因此，一个可行的 AOV 网络必须是 **DAG (Directed Acyclic Graph)**。

4 拓扑排序 (Topological Sort)

4.1 拓扑序的定义

拓扑序 (Topological Order) 是图中顶点的一种线性排列，使得对任意两个顶点 i, j，如果 i 是 j 的前驱，则在线性序列中 i 必须出现在 j 之前。

拓扑排序存在条件

一个有向图存在拓扑序，当且仅当它是 DAG。若图中有环，环上的每个顶点都要求另一个顶点先完成，排序无法开始或无法完成。

4.2 课程例子的一个拓扑序

拓扑序不一定唯一。对上面的课程先修图，课件给出的一种可行课程安排为：

Course number	Course name	Prerequisites
C1	Programming I	None
C2	Discrete Mathematics	None
C4	Calculus I	None
C3	Data Structure	C1, C2
C5	Calculus II	C4
C6	Linear Algebra	C5
C7	Analysis of Algorithms	C3, C6
C15	Numerical Analysis	C6
C8	Assembly Language	C3
C10	Programming Languages	C7
C9	Operating Systems	C7, C8
C12	Artificial Intelligence	C7
C13	Computational Theory	C7
C11	Compiler Design	C10
C14	Parallel Algorithms	C13

同一个 AOV 网络可能有多个拓扑序。例如同时没有先修课的课程之间可以互换顺序，只要不违反后续的先修关系即可。

4.3 拓扑排序目标

拓扑排序要完成两件事：

1. **测试 AOV 网络是否可行**：判断图中是否存在环。
2. **若可行，生成一个拓扑序**：输出顶点的线性次序。

4.4 朴素算法：每次重新寻找入度为 0 的顶点

朴素版本每一轮都扫描所有未编号顶点，寻找新的入度为 0 的顶点。

```
void Topsort( Graph G )
{
    int Counter;
    Vertex V, W;

    for ( Counter = 0; Counter < NumVertex; Counter++ ) {
        V = FindNewVertexOfDegreeZero();   /* O(|V|) */
        if ( V == NotAVertex ) {
            Error("Graph has a cycle");
            break;
        }
        TopNum[V] = Counter;               /* or output V */
        for ( each W adjacent from V )
            Indegree[W]--;
    }
}
```

复杂度分析：外层循环执行 $|V|$ 次，每次寻找入度为 0 的顶点可能扫描 $|V|$ 个顶点，因此

$$T = O(|V|^2).$$

4.5 改进算法：维护所有入度为 0 的顶点

改进思路是将所有尚未编号且入度为 0 的顶点放入一个“盒子”中，这个盒子可以是队列 (queue)，也可以是栈 (stack)。课件提示教材 Fig. 9.4 (p.289) 有误，复习时应以“删除顶点后更新其所有后继入度”为准。

课件右侧 v_1 到 v_7 示例

课件右侧示例图可整理为以下边集：

$$\begin{aligned} E = \{&v_1 \to v_2,\ v_1 \to v_3,\ v_1 \to v_4,\ v_2 \to v_4,\ v_2 \to v_5,\ v_3 \to v_6,\\ &v_4 \to v_3,\ v_4 \to v_6,\ v_4 \to v_7,\ v_5 \to v_4,\ v_5 \to v_7,\ v_7 \to v_6\}. \end{aligned}$$

其初始入度为：

$$\text{indegree}(v_1, \ldots, v_7) = (0, 1, 2, 3, 1, 3, 2).$$

若按队列方式处理，一个可能的删除次序为

$$v_1,\ v_2,\ v_5,\ v_4,\ v_3,\ v_7,\ v_6.$$

课件中入度表旁的连续数字表示：随着顶点被删除，相关后继的入度从 $3, 2, 1$ 逐步减到 0，一旦某个顶点入度变为 0，就可以进入队列等待输出。

```
void Topsort( Graph G )
{
    Queue Q;
    int Counter = 0;
    Vertex V, W;

    Q = CreateQueue( NumVertex );
    MakeEmpty( Q );

    for ( each vertex V )
        if ( Indegree[V] == 0 )
            Enqueue( V, Q );

    while ( !IsEmpty( Q ) ) {
        V = Dequeue( Q );
        TopNum[V] = ++Counter;              /* assign next */
        for ( each W adjacent from V )
            if ( --Indegree[W] == 0 )
                Enqueue( W, Q );
    }

    if ( Counter != NumVertex )
        Error("Graph has a cycle");

    DisposeQueue( Q );                      /* free memory */
}
```

若图用邻接表存储，每个顶点入队出队一次，每条边最多导致一次入度递减，因此：

$$T = O(|V| + |E|).$$

4.6 算法直觉与正确性

- **为什么从入度为 0 的顶点开始？**入度为 0 表示该活动没有尚未完成的前驱，可以立即安排。

- **为什么要递减后继入度？**输出顶点 V 等价于“完成活动 V”，因此所有以 V 为直接前驱的活动少了一个未完成前驱。
- **为什么能检测环？**如果队列为空但仍有顶点未输出，说明剩余顶点每个都有前驱在剩余集合内，必然形成有向环。
- **队列与栈的影响**：队列和栈都能得到合法拓扑序，但当存在多个入度为 0 的顶点时，它们可能输出不同的合法次序。

5 补充知识点与易错点

5.1 邻接矩阵与邻接表的选择

表示方式	优势	劣势
邻接矩阵	判断边是否存在为 $O(1)$；适合稠密图；矩阵表达直观	空间 $O(n^2)$；遍历边通常为 $O(n^2)$；稀疏图浪费空间
邻接表	空间 $O(n+e)$；遍历边为 $O(n+e)$；适合稀疏图	判断任意边是否存在通常需扫描链表；有向图求入度需额外结构或扫描

5.2 拓扑排序常考结论

1. 拓扑排序只针对**有向图**；无向图通常不谈拓扑序。
2. 有向图存在拓扑序 $\Longleftrightarrow$ 该图是 DAG。
3. 任意 DAG 至少存在一个入度为 0 的顶点，也至少存在一个出度为 0 的顶点。
4. 拓扑序不唯一；若每一步都只有一个入度为 0 的候选顶点，则 Kahn 算法得到的拓扑序是唯一的。
5. 自回路是长度为 1 的有向环，因此会直接破坏 AOV 网络的可行性。
6. 若输出顶点数少于 $|V|$，不要继续“随便选一个顶点”补全；这正是有环的证据。

5.3 DFS 版拓扑排序

除入度法外，也可以用深度优先搜索得到拓扑序：

- 对每个顶点做 DFS。
- 顶点的所有后继都处理完后，再把该顶点压入结果。
- 最后将结果逆序，即可得到拓扑序。

- 若 DFS 过程中遇到正在递归栈中的顶点，说明存在环。

DFS 版本同样可以做到 $O(|V|+|E|)$，常用于需要同时检测环和输出拓扑序的场景。

6 最短路径问题的基本概念

6.1 带权路径长度

给定有向图 $G=(V,E)$，以及每条边 $e \in E(G)$ 的代价函数 $c(e)$。从源点到终点的一条路径 P 的长度定义为路径上所有边权之和：

$$\text{length}(P) = \sum_{e_i \in P} c(e_i).$$

这个值也称为**带权路径长度 (weighted path length)**。

6.2 单源最短路径问题

Single-Source Shortest-Path Problem：输入带权图 $G=(V,E)$ 和一个指定源点 s，求从 s 到图中每一个其他顶点的最短带权路径。

关于负权边与负权环

- 边权可以为负，但若存在从源点可达的**负权环 (negative-cost cycle)**，最短路径没有有限答案，因为绕环越多总长度越小。
- 课件特别说明：如果不存在负权环，则从 s 到 s 的最短路径定义为 0。
- Dijkstra 算法不能处理负权边；后面要使用可重复松弛的算法。

6.3 表结构 Table

课件中的最短路径算法都围绕一个表 `Table` 展开。对每个顶点 v_i，维护：

字段	含义
`Table[i].Dist`	当前已知的从源点 s 到 v_i 的最短距离估计。初始化为 ∞，源点为 0。
`Table[i].Known`	若 v_i 已被确定并检查，则为 1；否则为 0。
`Table[i].Path`	用来记录最短路径上的前驱顶点。初始化为 0 或空。

6.4 松弛操作

最短路径算法的核心动作是**松弛 (relaxation)**。如果存在边 $\langle v,w\rangle$，并且

$$\text{Dist}[v] + c(v,w) < \text{Dist}[w],$$

则更新

$$\mathrm{Dist}[w] = \mathrm{Dist}[v] + c(v,w), \qquad \mathrm{Path}[w] = v.$$

直观地说：如果通过 v 到达 w 更短，就让 w 改用 v 作为前驱。

7 无权最短路径 (Unweighted Shortest Paths)

7.1 基本思想

无权图可以看作每条边权都为 1 的图。最短路径长度就是从源点出发经过的边数。课件例子以 v_3 为源点，按层次得到：

距离	顶点
0	v_3
1	v_1, v_6
2	v_2, v_4
3	v_5, v_7

这正是**广度优先搜索 (Breadth-First Search, BFS)** 的层次扩展过程。

7.2 简单实现

第一种实现按距离 $0, 1, 2, \ldots$ 逐层扫描整张表，找到所有未知且距离等于当前层数的顶点，再更新它们的邻接点。

```
void Unweighted( Table T )
{
    int CurrDist;
    Vertex V, W;

    for ( CurrDist = 0; CurrDist < NumVertex; CurrDist++ ) {
        for ( each vertex V )
            if ( !T[ V ].Known && T[ V ].Dist == CurrDist ) {
                T[ V ].Known = true;
                for ( each W adjacent to V )
                    if ( T[ W ].Dist == Infinity ) {
                        T[ W ].Dist = CurrDist + 1;
                        T[ W ].Path = V;
                    }
            }
    }
}
```

因为每一层都可能扫描所有顶点，最坏情况下复杂度为

$$T = O(|V|^2).$$

课件给出的最坏情形是一条长链：每一轮只找到一个新顶点，却仍要扫描整张顶点表。

7.3 队列改进实现

真正的 BFS 用队列保存刚被发现但尚未扩展的顶点，每个顶点最多入队一次，每条边最多检查一次。

```
void Unweighted( Table T )
{
    Queue Q;
    Vertex V, W;

    Q = CreateQueue( NumVertex );
    MakeEmpty( Q );
    Enqueue( S, Q );

    while ( !IsEmpty( Q ) ) {
        V = Dequeue( Q );
        T[ V ].Known = true;      /* not really necessary */

        for ( each W adjacent to V )
            if ( T[ W ].Dist == Infinity ) {
                T[ W ].Dist = T[ V ].Dist + 1;
                T[ W ].Path = V;
                Enqueue( W, Q );
            }
    }
    DisposeQueue( Q );
}
```

时间复杂度：

$$T = O(|V| + |E|).$$

7.4 课件例子的最终表

以 v_3 为源点，队列 BFS 得到：

顶点	Dist	Path
v_1	1	v_3
v_2	2	v_1
v_3	0	0
v_4	2	v_1
v_5	3	v_2
v_6	1	v_3
v_7	3	v_4

8 Dijkstra 算法

8.1 适用范围与集合定义

Dijkstra 算法用于**非负边权**的单源最短路径问题。设

$$S = \{s \text{ 以及已经确定最短路径的顶点}\}.$$

对于任意 $u \notin S$，定义 distance[u] 为所有形如

$$s \to (v_i \in S) \to u$$

的路径中的最小长度。

8.2 贪心选择规则

如果按非递减距离顺序生成路径，则每次选择满足

$$\text{distance}[u] = \min_{w \notin S} \text{distance}[w]$$

的未知顶点 u，将其加入 S。若最小顶点不唯一，任选一个。

Dijkstra 的贪心为什么成立

当所有边权非负时，最小未知距离顶点 u 不可能再通过其他未知顶点得到更短路径。因为一条绕到未知顶点再回来的路径至少不会比当前已知前缀更短。这个性质一旦遇到负权边就会失效。

8.3 算法伪代码

```
void Dijkstra( Table T )
{
    Vertex V, W;
```

```
    for ( ; ; ) {
        V = smallest unknown distance vertex;
        if ( V == NotAVertex )
            break;

        T[ V ].Known = true;

        for ( each W adjacent to V )
            if ( !T[ W ].Known )
                if ( T[ V ].Dist + Cvw < T[ W ].Dist ) {
                    Decrease( T[ W ].Dist to T[ V ].Dist + Cvw );
                    T[ W ].Path = V;
                }
    }
}
```

注意：课件在代码旁强调，Dijkstra **not work for edge with negative cost**。

8.4 课件例子的最终表

以 v_1 为源点，课件图中的最终最短路径表为：

顶点	Dist	Path	对应路径
v_1	0	0	v_1
v_2	2	v_1	$v_1 \to v_2$
v_3	3	v_4	$v_1 \to v_4 \to v_3$
v_4	1	v_1	$v_1 \to v_4$
v_5	3	v_4	$v_1 \to v_4 \to v_5$
v_6	6	v_7	$v_1 \to v_4 \to v_7 \to v_6$
v_7	5	v_4	$v_1 \to v_4 \to v_7$

8.5 路径打印

课件要求阅读教材 Figure 9.31 来打印路径。根据 `Path` 前驱数组，可以递归打印从源点到目标点的路径：

```
void PrintPath( Vertex V, Table T )
{
    if ( T[ V ].Path != NotAVertex ) {
        PrintPath( T[ V ].Path, T );
```

```
            printf( " to " );
        }
        printf( "%v", V );
    }
```

本质上是从目标点沿 Path 反向追溯到源点，再在递归返回时正向输出。

8.6 Dijkstra 的两种实现

实现方式	关键操作	复杂度与适用场景
扫描表	每次线性扫描所有未知顶点，找最小 Dist	$O(\|V\|^2 + \|E\|)$，适合稠密图
优先队列	用堆执行 DeleteMin，更新时执行 DecreaseKey 或重复插入	$O(\|E\| \log \|V\|)$，适合稀疏图

课件还提到：

- 若使用 DecreaseKey，每次调整距离为 $O(\log |V|)$。
- 若不做 DecreaseKey，可以把更新后的顶点再次插入优先队列；删除时反复 DeleteMin 直到遇到未知顶点。这会需要 $|E|$ 级别的删除次数和空间。
- 进一步改进可使用 Pairing Heap 或 Fibonacci Heap。

9 带负权边的图

9.1 为什么不能简单加常数

课件提出一个常见误区：如果存在负边，能不能给每条边都加同一个常数 D，让所有边都变成非负，再用 Dijkstra?

答案是不可以。原因是不同路径包含的边数可能不同。若每条边都加 D，一条含 k 条边的路径总长度会额外增加 kD。这样会改变不同路径之间的相对大小，最短路径可能被改掉。

9.2 队列式负权算法

课件给出一种简单但朴素的算法：只要某个顶点的距离被改小，就把它重新入队，继续用它松弛邻接点。

```
void WeightedNegative( Table T )
{
    Queue Q;
    Vertex V, W;

    Q = CreateQueue( NumVertex );
```

```
        MakeEmpty( Q );
        Enqueue( S, Q );

        while ( !IsEmpty( Q ) ) {
            V = Dequeue( Q );
            for ( each W adjacent to V )
                if ( T[ V ].Dist + Cvw < T[ W ].Dist ) {
                    T[ W ].Dist = T[ V ].Dist + Cvw;
                    T[ W ].Path = V;
                    if ( W is not already in Q )
                        Enqueue( W, Q );
                }
        }
        DisposeQueue( Q );
    }
```

9.3 复杂度与负权环

课件给出的时间复杂度为

$$T = O(|V| \cdot |E|).$$

理由是：没有负权环时，一个顶点最多可能被出队 $|V|$ 次左右，每次出队检查其邻接边。

与 Bellman-Ford 的关系

这个队列式写法可以看作 Bellman-Ford 思想的队列优化版本。Bellman-Ford 的标准形式对所有边重复松弛 $|V|-1$ 轮，复杂度也是 $O(|V||E|)$，并可通过第 $|V|$ 轮是否还能松弛来检测负权环。

如果存在负权环，课件代码会导致无限循环，因为某些 `Dist` 可以不断降低。

10 有向无环图中的最短路径

10.1 按拓扑序处理

若图是有向无环图 (DAG)，可以按照拓扑序选择顶点。原因是：当某顶点被处理时，不会再有来自未知顶点的入边把它的距离降低。

因此 DAG 最短路径不需要优先队列，只需按拓扑序进行一次松弛：

```
    for ( each vertex V in topological order )
        for ( each W adjacent to V )
            if ( Dist[V] + Cvw < Dist[W] ) {
                Dist[W] = Dist[V] + Cvw;
                Path[W] = V;
            }
```

总时间复杂度：

$$T = O(|V| + |E|).$$

DAG 最短路径允许负权边，只要无环即可。

10.2 AOE 网络

AOE (Activity On Edge) Network 用边表示活动，用顶点表示事件或活动完成的时间点，常用于项目调度。

- a_i：一项活动，对应一条有向边。
- 边权 $C_{v,w}$：活动持续时间 (lasting time)。
- $EC[j]$：顶点 v_j 的最早完成时间 (earliest completion time)。
- $LC[j]$：顶点 v_j 的最晚完成时间 (latest completion time)。
- CPM：Critical Path Method，关键路径法。

10.3 EC 与 LC 的计算

从起点 v_0 开始，按照拓扑序计算最早完成时间：

$$EC[w] = \max_{(v,w)\in E}\{EC[v] + C_{v,w}\}.$$

从最后一个顶点开始，按照逆拓扑序计算最晚完成时间：

$$LC[v] = \min_{(v,w)\in E}\{LC[w] - C_{v,w}\}.$$

活动 $\langle v, w\rangle$ 的松弛时间为：

$$\text{Slack}(v, w) = LC[w] - EC[v] - C_{v,w}.$$

完全由零松弛边组成的路径就是**关键路径 (critical path)**。关键路径长度决定项目最短完成时间。

10.4 课件 AOE 例题

课件的项目网络共有顶点 0 到 8，活动如下：

活动	边	持续时间
a_0	$0 \to 1$	6
a_1	$0 \to 2$	4
a_2	$0 \to 3$	5
a_3	$1 \to 4$	1
a_4	$2 \to 4$	1
a_5	$3 \to 5$	2
a_6	$4 \to 6$	9
a_7	$4 \to 7$	7
a_8	$5 \to 7$	4
a_9	$6 \to 8$	2
a_{10}	$7 \to 8$	4
a_{11}	$5 \to 4$	0，dummy activity

顶点上的最早和最晚完成时间为：

顶点	0	1	2	3	4	5	6	7	8
EC	0	6	4	5	7	7	16	14	18
LC	0	6	6	5	7	7	16	14	18

若计算活动松弛时间，可得例如：

- $a_1 : 0 \to 2$ 的松弛为 $LC[2] - EC[0] - 4 = 6 - 0 - 4 = 2$。
- $a_4 : 2 \to 4$ 的松弛为 $LC[4] - EC[2] - 1 = 7 - 4 - 1 = 2$。
- $a_8 : 5 \to 7$ 的松弛为 $LC[7] - EC[5] - 4 = 14 - 7 - 4 = 3$。

项目最短完成时间为 $EC[8] = 18$。零松弛边可形成关键路径，例如：

$$0 \to 1 \to 4 \to 6 \to 8,$$

其总持续时间为 $6 + 1 + 9 + 2 = 18$。

11 全源最短路径问题

11.1 问题定义

All-Pairs Shortest Path Problem：对所有顶点对 v_i, v_j，其中 $i \neq j$，求 v_i 到 v_j 的最短路径。

11.2 课件给出的两种方法

方法	思想	课件说明
方法 1	对每个顶点都运行一次单源最短路径算法	$O(\|V\|^3)$，在稀疏图上工作较快
方法 2	使用第 10 章给出的 $O(\|V\|^3)$ 算法	同阶复杂度，但在稠密图上更快

这里第 10 章的算法通常指 Floyd-Warshall 思想：逐步允许更多中间顶点参与路径，动态更新任意两点之间的最短距离。

12 最短路径算法选择总结

图的条件	推荐算法	复杂度
无权图	BFS 队列实现	$O(\|V\|+\|E\|)$
非负带权图，稠密	Dijkstra 扫描表	$O(\|V\|^2+\|E\|)$
非负带权图，稀疏	Dijkstra + 优先队列	$O(\|E\|\log\|V\|)$
含负权边但无负权环	Bellman-Ford 或队列式松弛	$O(\|V\|\|E\|)$
有向无环图	拓扑序松弛	$O(\|V\|+\|E\|)$
全源最短路径	多次单源或 Floyd-Warshall	$O(\|V\|^3)$

最短路径复习抓手

1. 先判断图是否无权、是否有负权边、是否有环。
2. 所有最短路径算法都在做松弛，只是选择下一个顶点的策略不同。
3. Dijkstra 的正确性依赖非负边权；DAG 算法依赖拓扑序；Bellman-Ford 依赖反复松弛。
4. `Path` 数组记录前驱，用来从终点倒推并打印完整路径。
5. AOE/CPM 题目重点是 EC、LC、Slack 和 zero-slack critical path。

13 网络流问题 (Network Flow Problems)

13.1 最大流模型

课件用管道网络引入最大流问题。网络中有一个源点 s (source) 和一个汇点 t (sink)，每条有向边都有容量 (capacity)。目标是确定从 s 到 t 最多能传送多少流量。

课件例子的容量网络为：

边	容量
$s \to a$	3
$s \to b$	2
$a \to b$	1
$a \to c$	3
$a \to d$	4
$b \to d$	2
$c \to t$	2
$d \to t$	3

对任意中间顶点 $v \notin \{s, t\}$，必须满足**流量守恒**：

$$\text{total flow coming into } v = \text{total flow going out of } v.$$

源点只产生流，汇点只接收流。

13.2 增广路算法的基本版本

课件先给出一个简单算法。维护当前流图 G_f 和残量图 G_r：

1. 在残量图 G_r 中找任意一条从 s 到 t 的路径。
2. 取这条路径上最小的残量容量作为本次可增加的流量。
3. 将该流量加入流图 G_f，并更新残量图 G_r，移除残量为 0 的边。
4. 如果 G_r 中仍存在 $s \to t$ 的路径，则回到第 1 步；否则结束。

这样的 $s \to t$ 路径称为**增广路 (augmenting path)**。

为什么“任意选择路径”会有问题

课件提醒：此时不能简单贪心。若过早选择了不好的路径，例如把流量送到某条瓶颈边上，后面可能需要撤销部分选择，才能得到最大流。因此残量图必须允许算法“undo its decisions”。

13.3 残量图与反向边

为允许撤销已有选择，课件给出关键修正：

若流图 G_f 中边 (v, w) 当前流量为 $f_{v,w}$，则在残量图 G_r 中加入反向边 (w, v)，容量为 $f_{v,w}$。

反向边的含义是：可以把已经经过 (v, w) 的一部分流退回，从而让后续增广路重新分配流量。

Ford-Fulkerson 思想

“找增广路 + 沿路径增加流 + 用残量图支持撤销”就是 Ford-Fulkerson 方法的核心。课件命题指出：若所有边容量为有理数，该算法总会终止并得到最大流。算法也可以处理带环的图。

13.4 整数容量下的复杂度

若容量全为整数，则每次增广至少增加 1 单位流。设最大流值为 f，若每次用无权最短路径算法在残量图中找一条增广路，则：

$$T = O(f \cdot |E|).$$

这个界依赖最大流值 f，当容量数值很大时可能很差。课件的反例结构是：

边	容量
$s \to a$	1,000,000
$s \to b$	1,000,000
$a \to b$	1
$a \to t$	1,000,000
$b \to t$	1,000,000

若反复选到经过 $a \to b$ 的瓶颈增广路，就会导致增广次数过多。

13.5 选择增广路的改进策略

课件给出两类选择增广路的策略：

1. **总是选择能带来最大流量增加的增广路**：可修改 Dijkstra 算法，目标不是最短距离，而是最大化路径瓶颈容量。

$$T = T_{\text{augmentation}} \cdot T_{\text{find a path}} = O(|E| \log cap_{\max}) \cdot O(|E| \log |V|).$$

若 $cap_{\max}$ 是较小整数，则课件写成

$$T = O(|E|^2 \log |V|).$$

2. **总是选择边数最少的增广路**：用无权最短路径算法，即 BFS，在残量图中找从 s 到 t 的最短边数路径。

$$T = T_{\text{augmentation}} \cdot T_{\text{find a path}} = O(|E|) \cdot O(|E| \cdot |V|) = O(|E|^2|V|).$$

第二种策略就是 Edmonds-Karp 的思想：固定用 BFS 找最短增广路，从而避免任意选路导致的坏情况。

13.6 补充说明：特殊容量与最小费用流

课件最后给出两个说明：

- 若每个 $v \notin \{s,t\}$ 都只有一条容量为 1 的入边，或只有一条容量为 1 的出边，则时间界可降为
$$O(|E||V|^{1/2}).$$
- **最小费用流 (min-cost flow)**：在所有最大流中，若每条边还有单位流费用，则要找总费用最小的那个最大流。

最大流复习抓手

- 可行流要满足容量限制和中间点流量守恒。
- 残量图不仅记录剩余容量，也记录可撤销已有流量的反向边。
- 算法停止条件是残量图中不存在从 s 到 t 的增广路。
- 找增广路的策略决定复杂度：任意路径可能很慢，最短增广路可得到 $O(|E|^2|V|)$。

14 最小生成树 (Minimum Spanning Tree)

14.1 生成树与最小生成树

Spanning tree：图 G 的一棵生成树由 $V(G)$ 的全部顶点和 $E(G)$ 的一个子集构成，并且整体是一棵树。

Minimum Spanning Tree, MST：在所有生成树中，总边权最小的生成树。

MST 的基本性质

- 它是树，因此无环，边数为 $|V|-1$。
- 它是 minimum，因为边权总和最小。
- 它是 spanning，因为覆盖每一个顶点。
- 最小生成树存在当且仅当图 G 连通。
- 向一棵生成树加入一条非树边，一定会得到一个环。

14.2 贪心方法的约束

最小生成树算法都属于贪心方法：每个阶段做当前最好的选择，但必须遵守：

1. 只能使用图中已有的边。

2. 最终必须恰好使用 $|V|-1$ 条边。

3. 不能加入会产生环的边。

14.3 Prim 算法：grow a tree

Prim 算法从一个顶点出发，不断把一条连接“树内顶点”和“树外顶点”的最小边加入树中。课件说明它与 Dijkstra 算法非常相似。

两者区别在于：

- Dijkstra 的 `Dist[v]` 表示从源点到 v 的当前最短路径长度。
- Prim 的 `Dist[v]` 表示把 v 接入当前生成树所需的最小边权。

```
while ( there is an unknown vertex ) {
    V = unknown vertex with smallest Dist;
    mark V known;
    for ( each W adjacent to V )
        if ( !Known[W] && Cvw < Dist[W] ) {
            Dist[W] = Cvw;
            Parent[W] = V;
        }
}
```

课件图中的 MST 可选边为：

边	权重
$v_1 - v_4$	1
$v_6 - v_7$	1
$v_1 - v_2$	2
$v_3 - v_4$	2
$v_4 - v_7$	4
$v_7 - v_5$	6

总权重为 $1+1+2+2+4+6=16$。

朴素扫描实现复杂度为 $O(|V|^2)$；若用优先队列维护候选边，可做到 $O(|E|\log|V|)$ 级别。

14.4 Kruskal 算法：maintain a forest

Kruskal 算法从空边集开始，每次选择当前权重最小的边。若加入它不会形成环，就保留；否则丢弃。整个过程中维护的是一片森林，直到森林合并为一棵生成树。

```
void Kruskal( Graph G )
{
```

```
    T = {};
    while ( T contains less than |V| - 1 edges
            && E is not empty ) {
        choose a least cost edge (v, w) from E;  /* DeleteMin */
        delete (v, w) from E;
        if ( (v, w) does not create a cycle in T )
            add (v, w) to T;                     /* Union / Find */
        else
            discard (v, w);
    }
    if ( T contains fewer than |V| - 1 edges )
        Error( "No spanning tree" );
}
```

课件给出的复杂度为：

$$T = O(|E| \log |E|).$$

原因是边需要按权重排序或用优先队列反复 `DeleteMin`；判断是否成环可用 Union-Find 维护连通分量。

14.5 Prim 与 Kruskal 对比

算法	核心思想	适用直觉
Prim	从一个顶点开始，每次选连接当前树和外部顶点的最小边	更像 Dijkstra，常适合稠密图或邻接矩阵实现
Kruskal	全局按边权从小到大考虑，跳过会成环的边	更依赖边排序和 Union-Find，常适合稀疏图

MST 复习抓手

- MST 是无向连通带权图上的问题；若图不连通，只能得到生成森林。
- Prim 防止“树外断开”，Kruskal 防止“加入边成环”。
- 加一条非树边会形成环；删掉环中一条边可恢复树结构。
- 边权不唯一时，MST 可能不唯一，但最小总权重相同。

15 深度优先搜索的应用 (Applications of DFS)

15.1 DFS 模板

深度优先搜索是前序遍历思想在图上的推广：

```
void DFS ( Vertex V )   /* 仅为模板 */
{
    visited[ V ] = true;            /* 标记，避免重复访问 */
    for ( each W adjacent to V )
        if ( !visited[ W ] )
            DFS( W );
}
/* 使用邻接表时 T = O( |E| + |V| ) */
```

注意： DFS 从单个起点出发。若图不连通，需要对每个未访问顶点都调用一次 DFS，才能遍历整个图。

15.2 利用 DFS 求连通分量 (Finding Connected Components)

若无向图 G 不连通，它由若干个**连通分量**（极大连通子图）组成。DFS 可以在 $O(|V|+|E|)$ 时间内找出全部连通分量。

算法思想

维护 `visited[]` 数组标记已访问顶点。对每个尚未访问的顶点调用一次 DFS，每次调用将访问并标记完整的一个连通分量内的所有顶点：

```
for ( each vertex V )
    if ( !visited[ V ] )
        DFS( V );
```

调用 `DFS` 的总次数即为图中连通分量的个数。每次调用所访问到的顶点集合就是一个连通分量。

深度优先生成森林 (Depth-First Spanning Forest)

对整图执行上述遍历，所有调用过程中走过的树边共同构成一片**深度优先生成森林**，其中每棵树恰好对应一个连通分量。

复杂度说明

无论图有多少个连通分量，总时间始终为

$$T = O(|V| + |E|)$$

（使用邻接表）。这是因为每个顶点和每条边在整个遍历过程中至多被访问一次。

课件示例

课件以顶点 0 为起点调用 `DFS(0)`，访问与 0 连通的所有顶点，完成第一个连通分量的标记。若图中仍有未访问顶点，则继续调用下一次 DFS，直到所有顶点均被访问。

15.3 无向图中的 DFS (Undirected Graphs)

对无向连通图执行 DFS，遍历过程中所经历的树边构成一棵**深度优先生成树 (Depth-First Spanning Tree)**。非树边（称为 back edges）都连接祖先和后代，不会出现交叉边。

对顶点按 DFS 访问顺序编号，得到**深度优先编号 Num**(v)。性质：

- 若 u 是 v 的祖先，则 $\mathrm{Num}(u) < \mathrm{Num}(v)$。
- 非树边 (u, v) 中，u 和 v 必为祖先-后代关系。

15.4 双连通性 (Biconnectivity)

概念定义

- **关节点 (Articulation Point)**：无向连通图中，删除该顶点（及其所有关联边）后，图变得不连通，则称该顶点为关节点。
- **双连通图 (Biconnected Graph)**：没有关节点的连通图。即删除任意一个顶点后图仍然连通。
- **双连通分量 (Biconnected Component)**：图的一个极大双连通子图。注意：两个双连通分量之间不能共享边（但可以共享关节点）。$E(G)$ 被双连通分量划分（partitioned）。

用 DFS 求关节点

利用 DFS 编号 $\mathrm{Num}(v)$ 和 $\mathrm{Low}(v)$ 函数来找关节点。

定义 $\mathrm{Low}(v)$ 为从以 v 为根的子树出发，通过至多一条 back edge 能到达的最小 DFS 编号：

$$\mathrm{Low}(v) = \min\Big\{\mathrm{Num}(v),\ \min_{(v,w)\in \text{back edges}} \mathrm{Num}(w),\ \min_{\text{child } c \text{ of } v} \mathrm{Low}(c)\Big\}.$$

判断关节点的规则：

1. **根结点**：若 DFS 树的根有两棵或两棵以上子树（即两个以上孩子），则根是关节点。
2. **非根结点** u：若 u 有某个孩子 v，使得 $\mathrm{Low}(v) \geq \mathrm{Num}(u)$，则 u 是关节点。（意味着 v 的子树不借助 back edge 就无法绕过 u 到达 u 的祖先。）

Low 值的直觉

$\mathrm{Low}(v)$ 衡量 v 的子树能” 退回” 到多浅的祖先。若 v 的子树完全无法通过 back edge 绕过父亲 u（即 $\mathrm{Low}(v) \geq \mathrm{Num}(u)$），则删掉 u 会切断 v 的子树与上层的联系，u 就是关节点。

课件提示：详细伪代码见教材 p.327 和 p.329。时间复杂度为 $O(|E| + |V|)$。

15.5 欧拉回路 (Euler Circuits)

定义与存在条件

欧拉回路 (Euler Circuit)：经过图中每条边恰好一次且最终回到起点的回路。

欧拉回路存在定理

连通无向图存在欧拉回路，当且仅当**每个顶点的度数均为偶数**。
有向图存在欧拉回路，当且仅当图弱连通且每个顶点的入度等于出度。

欧拉路径 (Euler Path)（不需回到起点）存在的条件：恰好有 0 或 2 个奇度顶点（2 个时以奇度顶点为起终点）。

算法思路

用 DFS 构造欧拉回路：

```
从起点出发，沿边 DFS 走，走过的边标记为已用。
每次走到死路（当前顶点无可用边）时，把该顶点加入路径尾部。
最终逆序输出路径即为欧拉回路。
```

实现要点（课件强调）

- **路径用链表维护**：频繁在中间插入顶点，链表比数组高效。
- **每条邻接表维护一个"当前扫描指针"**：避免每次都从头扫描已用过的边，使总复杂度保持 $O(|E|+|V|)$。

16 全章总结 (Summary)

本讲义把第九章图相关内容整合在一起：前半覆盖图的定义、图的表示方法、AOV 网络与拓扑排序；后半覆盖无权最短路径、Dijkstra、负权边、DAG 最短路径、AOE/CPM、全源最短路径、网络流与最小生成树。复习时可以抓住以下主线：

1. **图由顶点集和边集组成**：无向边没有方向，有向边有 tail 和 head；本章默认不考虑自回路和重图。
2. **路径与连通性是图算法的语言基础**：路径、简单路径、环、连通图、连通分量、强连通与弱连通都在描述顶点之间的可达关系。
3. **度数要按图类型区分**：无向图只谈度；有向图要区分入度、出度和总度。
4. **存储方式影响算法复杂度**：邻接矩阵适合稠密图，邻接表适合稀疏图；拓扑排序使用邻接表可以达到 $O(|V|+|E|)$。

5. **AOV 网络表达先后关系**：顶点表示活动，边表示优先关系；可行 AOV 网络必须是 DAG。
6. **拓扑排序的本质是不断删除入度为 0 的顶点**：若能删除完所有顶点，则得到拓扑序；若删除不完，则图中有环。
7. **最短路径的核心是松弛**：无权图用 BFS，非负带权图用 Dijkstra，负权边用 Bellman-Ford 思想，DAG 可直接按拓扑序处理。
8. **关键路径是项目调度中的最长路径问题**：先算 EC，再逆向算 LC，松弛时间为 0 的边组成关键路径。
9. **最大流的核心是增广路与残量图**：残量图中的反向边允许撤销已有决策；没有 $s \to t$ 增广路时算法停止。
10. **最小生成树的核心是安全选边**：Prim 每次扩展当前树，Kruskal 每次选择全局最小且不成环的边。
11. **DFS 可求连通分量，也可生成 DFS 编号和 Low 值**：对每个未访问顶点调用一次 DFS 即可枚举所有连通分量；$\text{Low}(v) \geq \text{Num}(u)$ 是 u 为关节点（非根）的判据。
12. **欧拉回路存在当且仅当所有顶点偶度**：算法用 DFS 结合链表路径维护，$O(|E| + |V|)$。

“图算法的第一步，是把对象和关系准确翻译成顶点与边；第二步，是选择适合图结构的遍历、松弛、增广或选边策略。”